Versión 0.8 — 3 de octubre de 2026

Unidades: 0.1, 0.2, 0.3 y 1.0 (subitizing)

Novedad de esta versión: la unidad 1.0 — reconocer de un vistazo

Ya se puede enseñar una figura y preguntar cuántos elementos tiene, **de un vistazo**:

```
subitizar: x.x/.../x.x

FIGURA:
    x . x
    . . .
    x . x
  es el patron 'esquinas_cuatro'  ->  CUATRO
RESULTADO: reconocido
```

La figura se escribe con **tres filas separadas por** `/`, donde `x` es un elemento y `.` un hueco.

Lo importante: el sistema no cuenta

Y aquí está la gracia de la unidad. Si la figura **no** es un patrón conocido:

```
subitizar: x../xx./x.x

FIGURA:
    x . .
    x x .
    x . x
RESULTADO: no_reconocido
  No lo reconozco de un vistazo, y NO lo cuento: contar es la unidad 2.3.
  El sistema no adivina: si no lo reconoce, lo dice.
```

Esa figura tiene **cuatro** elementos. El sistema podría contarlos… si supiera contar, y todavía no sabe **(eso es el Nivel 2). Así que no lo finge: dice que no lo reconoce.** Reconocer no es contar.

Y la disposición es lo que hace reconocible a un grupo

- `x.x/.../x.x` → cuatro (esquinas, como un dado) ✔
- `xx./xx./...` → cuatro también: la misma cantidad en otra disposición vale lo mismo ✔
- `x../xx./x.x` → no se reconoce ✘

Esto no es un capricho: la investigación muestra que las disposiciones tipo dado se ven **enteras** y las desordenadas no, y que una fila recta de puntos es difícil de subitizar.

Las cantidades van en **palabras** (`uno`, `cuatro`), no en cifras: las cifras son la unidad 3.1.

Novedad de esta versión: el mismo nombre, dos objetos

Corregido a raíz de una prueba tuya. Antes, esto:

```
carro(color=negro), carro(color=blanco)
```

decía «es **un** objeto» y «se contradicen». Estaba mal, como tú dijiste: una misma cosa no puede ser negra y blanca a la vez**, así que un atributo con** dos valores distintos **es la** prueba de que son **dos objetos**. Ahora:

```
   AVISO: un mismo nombre con datos que NO pueden ser la misma cosa:
     carro  [color=negro]
     carro  [color=blanco]
   -> eso PRUEBA que son objetos DISTINTOS. Quedan asi: carro#1, carro#2
PLURALIDAD (0.1): son VARIOS OBJETOS
ATRIBUTOS (0.2): sin_atributo_comun
```

Las tres reglas del nombre, ya juntas:

Escribes	Qué entiende
`casa(color=blanco); casa(forma=cuadrada)`	**La misma casa** (los datos se suman): `casa` blanca y cuadrada
`carro(color=negro); carro(color=blanco)`	**Dos objetos**: `carro#1` y `carro#2` — el color los distingue
`carro, carro`	**Un solo objeto**: sin atributos, nada los distingue

Novedad de esta versión: la unidad 0.3 — pertenencia

Ya se puede preguntar **si un objeto concreto pertenece a un grupo**:

```
pertenece color=rojo: taza(color=rojo); plato(color=azul); vaso(forma=cubo)

  taza       -> PERTENECE
  plato      -> NO_PERTENECE
  vaso       -> NO_DECIDIBLE  (no se sabe su color)
```

Tres respuestas, no dos, y esa es toda la gracia:

- **PERTENECE** — el objeto tiene el color que pide el grupo.
- **NO_PERTENECE** — tiene color, pero **otro**.
- **NO_DECIDIBLE** — **no se sabe**: a ese objeto no le hemos dicho el color. Que no es lo mismo que «no pertenece»: para negar algo hay que saber que es falso, no basta con no saberlo.

Y algo que conviene probar: **un mismo objeto pertenece a varios grupos**. El triángulo verde pertenece a «los verdes» y a «los triángulos» — la pertenencia depende del criterio con el que se pregunta.

El Nivel 0 está completo

Con 0.1, 0.2 y 0.3 cerradas y verificadas, el sistema ya sabe: distinguir objetos, formar grupos por atributos y decidir pertenencia. **Todo rastreable y todo con casos nuevos probados.**

Novedad de esta versión: cada respuesta dice las DOS cosas

Cuando escribes objetos con atributos, el sistema contesta **en dos capas**:

- **PLURALIDAD (0.1):** cuántas cosas hay — `es UN OBJETO` o `son VARIOS OBJETOS`.
- **ATRIBUTOS (0.2):** si comparten algún atributo, y cuáles.

Así ya no esconde lo que sabe: `casa(color=azul); carro` **no** dice un «indefinido» a secas, sino «son **varios objetos**, y en cuanto a atributos **no se puede decidir** (del color del carro no sabemos nada)».

Y hay una regla nueva, la de la **fusión**:

Escribes	Qué entiende
`casa(color=blanco); casa(forma=cuadrada)`	**La misma casa**, descrita dos veces: sus datos se suman → `casa blanca y cuadrada`, **un solo objeto**
`casa(color=blanco); casa(color=rojo)`	**Se contradicen** (mismo atributo, dos valores) → no elige: `no_decidible`
`casa, casa`	**Un solo objeto** (el nombre manda) — si no, sería un grupo y eso rompería 0.1

Novedad de esta versión: el vocabulario, arreglado

Carlos destapó una contradicción: `casa,carro` decía *grupo*, y `casa(color=blanco); carro(color=negro)` decía *no grupo*. La misma pareja de objetos, respuestas opuestas.

La causa era usar la palabra **grupo** con dos sentidos distintos. Ya está separado:

Unidad	La pregunta que responde	Respuestas
0.1	¿hay **una** cosa o **varias**? (sin contar)	`un_objeto` / `varios_objetos`
0.2	¿esos objetos **comparten** algo?	`grupo_por_atributo` / `sin_atributo_comun` / `una_sola_entidad`

La palabra GRUPO es solo de 0.2 y siempre quiere decir: varios objetos que comparten un atributo con el mismo valor. Así ya no se contradicen:

```
casa,carro                                -> varios_objetos
casa(color=blanco); carro(color=negro)    -> varios objetos, pero sin_atributo_comun
casa_A(color=blanco); casa_B(color=blanco)  -> grupo_por_atributo, comparten color
```

En una línea

Doble clic en `E:\Euclides\PROBAR.bat`. Se abre una ventana, corre tus casos y te deja escribir los

tuyos. Para salir, escribe `salir`.

Qué sabe hacer hoy (y qué no)

0.1	Decidir si hay un objeto o varios objetos (sin contarlos)
0.2	**Reconocer** si varios objetos forman grupo y decir **qué atributos comparten**
0.2	**Formar** el grupo de los que cumplen un criterio (`color=rojo`)
0.3	**Pertenencia**: si un objeto concreto pertenece a un grupo (`pertenece` / `no_pertenece` / `no se sabe`)
1.0	**Subitizing**: reconocer de un vistazo la cantidad de un grupo pequeño, sin contar
Siempre	Contesta con **las dos capas**: cuántas cosas hay y si comparten atributos
Siempre	Si no entiende lo que le escribes, **lo dice**; nunca revienta
No sabe todavía	Contar. No dice "hay tres". No compara cantidades

Las tres formas de preguntar

1. Un objeto o varios (unidad 0.1)

Cosas separadas por **comas**. Si repites un nombre, es **la misma** cosa señalada otra vez.

```
silla                -> un_objeto
silla,mesa           -> varios_objetos
silla,silla          -> un_objeto   (la misma silla, dos veces)
```

2. ¿Forman grupo? (unidad 0.2)

Objetos con atributos. Valen **punto y coma** o **comas de fuera** de los paréntesis. Los atributos van entre paréntesis como `nombre=valor`, separados por comas.

```
caja_A(color=rojo,forma=cubo); caja_B(color=rojo,forma=esfera)   -> grupo por color
caja_A(color=rojo,forma=cubo), caja_B(color=azul,forma=cubo)     -> grupo por forma
casa(color=blanco); carro(color=negro)                           -> sin_atributo_comun
```

3. Dame el grupo de... (unidad 0.2)

El criterio, dos puntos, y la escena.

```
color=rojo: a(color=rojo,forma=cubo); b(color=azul,forma=cubo); c(color=rojo,forma=esfera)
   -> el grupo son a y c
```

4. ¿Pertenece? (unidad 0.3)

Empieza por `pertenece`, luego el criterio del grupo, dos puntos, y el objeto o los objetos.

```
pertenece color=rojo: taza(color=rojo)
   -> taza -> PERTENECE

pertenece color=rojo: taza(color=rojo); plato(color=azul); vaso(forma=cubo)
   -> taza  -> PERTENECE
   -> plato -> NO_PERTENECE
   -> vaso  -> NO_DECIDIBLE  (no se sabe su color)
```

5. ¿Cuántos hay, de un vistazo? (unidad 1.0)

Empieza por `subitizar`, dos puntos, y la figura: **tres filas separadas por** `/`, con `x` de elemento y `.` de hueco.

```
subitizar: x.x/.../x.x
   -> es el patron 'esquinas_cuatro' -> CUATRO

subitizar: x../xx./x.x
   -> no_reconocido  (no lo reconoce de un vistazo, y NO lo cuenta)
```

Lo más importante: el nombre es el identificador

El sistema **no ve** los objetos, y **no adivina** por el parecido de los nombres. El nombre que escribes **es** la identidad de la cosa.

- `silla` y `silla` → **la misma** silla.
- `silla_A` y `silla_B` → **dos** sillas distintas, aunque en el mundo las dos se llamen igual.

Y si escribes la misma cosa con dos descripciones que no pueden ser ciertas a la vez:

```
casa(color=azul); casa(color=rojo)
   -> son DOS objetos distintos: el color los distingue (casa#1 y casa#2)
```

Pero si los datos **no** se contradicen, son la misma cosa y sus datos se **suman**:

```
casa(color=blanco); casa(forma=cuadrada)
   -> un solo objeto: casa blanca y cuadrada
```

Si escribes algo que no entiende

No revienta: te lo dice y te recuerda las formas correctas.

```
pregunta> caja(colorrojo)

NO ENTIENDO ESTO: no entiendo el atributo 'colorrojo'
   se escribe asi:  nombre=valor

FORMAS QUE SI ENTIENDO:
  0.1  un objeto o varios  silla,mesa
  0.2  forman grupo?       a(color=rojo,forma=cubo); b(color=azul,forma=cubo)
  0.2  dame el grupo de... color=rojo: a(color=rojo); b(color=azul)

El sistema no adivina: si no entiende, lo dice.
```

Las tres formas de lanzarlo

1. **Doble clic en** `PROBAR.bat` — lo normal. Corre tus casos y abre la ventana.
2. `py -3 E:\Euclides\probar.py "silla,mesa"` — una pregunta suelta.
3. `py -3 E:\Euclides\probar.py --casos` — todos tus casos de golpe.

Órdenes dentro de la ventana: `casos`, `ayuda`, `salir`.

Dónde escribes TUS casos

Archivo: `E:\Euclides\mis_casos.txt` (Bloc de notas). Cada línea:

```
pregunta | lo que espero | comentario
```

Lo que puedes poner en "lo que espero":

Escribes	Significa
`objeto` o `uno`	una sola entidad
`varios`	dos o más entidades distintas
`grupo:color`	forman grupo y comparten el color
`grupo:color,forma`	comparten las dos cosas
`sin_atributo_comun`	son varios, pero no comparten nada
`una_sola_entidad`	solo hay una cosa (o es la misma descrita dos veces): la pregunta de grupo no aplica
`tiene:a,c`	al formar un grupo, el resultado son esos
`no_hay_grupo`	ninguno cumple el criterio
`no_decidible`	el sistema debe decir que no sabe
`no_entendido`	el sistema debe decir que no entiende la escritura
`taza:pertenece`	(0.3) la taza pertenece al grupo que se pregunta
`plato:no_pertenece`	(0.3) el plato no pertenece
`vaso:no_decidible`	(0.3) del vaso no se sabe: no es lo mismo que no pertenecer
`reconocido:cuatro`	(1.0) la figura es el patrón de cuatro
`no_reconocido`	(1.0) el sistema no reconoce la figura de un vistazo, y no la cuenta
?	no comprobar nada, solo ver qué responde

Guarda el archivo y vuelve a lanzar el probador: **se recarga solo**. Los # son comentarios.

Por qué el sistema explica lo que dice

Cada respuesta sale con su **traza**: qué reglas se han probado, en qué orden, cuál ha decidido y qué conocimiento escrito la respalda. Ejemplo real:

```
PREGUNTA (0.2): forman GRUPO estos objetos?
```

```
ESCENA:
    casa  [color=blanco]
    carro [color=negro]
  reglas probadas, en orden:
    - reconocer_escena_vacia : escena_vacia(escena) -> falso
    - reconocer_entidad_contradictoria : hay_entidad_contradictoria(escena) -> falso
    - reconocer_una_sola_entidad : menos_de_dos_entidades(escena) -> falso
    - reconocer_atributo_indeterminado : algun_atributo_indeterminado(escena) -> falso
    - reconocer_grupo_por_atributo : hay_atributo_compartido_por_todos(escena) -> falso
    - reconocer_sin_atributo_comun : sin_atributo_compartido_por_todos(escena) -> cierto
RESULTADO: sin_atributo_comun
  lo decide la regla 'reconocer_sin_atributo_comun'
  respaldado por el conocimiento 'grupo_por_atributo' [FIJADO]
```

Qué buscar cuando lo pruebes

1. **Que 0.1 y 0.2 ya no se contradigan**: `casa,carro` contra

`casa(color=blanco); carro(color=negro)`.

2. **La pertenencia y sus tres respuestas**: `pertenece color=rojo: taza(color=rojo); plato(color=azul); vaso(forma=cubo)`.

Comprueba que el vaso dice **«no se sabe»** y no «no pertenece».

3. **El mismo objeto en dos grupos**: `pertenece color=verde:` y `pertenece forma=triangulo:` sobre

`triangulo(color=verde,forma=triangulo)`. Debería pertenecer a los dos.

4. **La fusión y la distinción**: `casa(color=blanco); casa(forma=cuadrada)` debería decir que es la

misma casa **con los datos sumados; y** `carro(color=negro); carro(color=blanco)` **debería dar** dos

objetos (`carro#1`, `carro#2`), no «no se sabe».

5. **Cosas escritas a medias o mal** (`caja(colorrojo)`, `caja(color=)`, `pertenece =rojo: ...`): todas

deberían contestar "no entiendo", ninguna debería romperse.

6. **La misma cosa con dos descripciones distintas que sí chocan** (`casa(color=azul); casa(color=rojo)`).

7. **Un objeto al que le falta un atributo** que los demás sí tienen: debería callarse **en atributos**

pero seguir diciendo que son varios objetos.

8. **Repetir el mismo nombre** en 0.1 (`taza,taza,taza`).

9. **Pedir un grupo que no existe** (`color=verde:` cuando nadie es verde).

10. **El subitizing y su negativa**: `subitizar: x.x/.../x.x` debería dar **cuatro**; y

`subitizar: x../xx./x.x` (que también tiene cuatro) debería dar **no_reconocido** — porque el

sistema **no cuenta**. Ese "no" es la respuesta correcta, no un fallo.

11. Cosas tuyas: nombres con espacios, mayúsculas y minúsculas, números (`caja1,caja2`).

Si algo te chirría, dímelo con **la línea exacta que escribiste y lo que esperabas**.

Los archivos, por si quieres mirar

Archivo	Qué es
`E:\Euclides\conocimiento\*.kn`	El saber, en texto plano. Se puede leer y editar
`E:\Euclides\sistema\motor.py`	El programa que lee, decide y explica
`E:\Euclides\pruebas\`	Las pruebas del proyecto (0.1, 0.2 y robustez)
`E:\Euclides\mis_casos.txt`	**Tus casos**
`E:\Euclides\PROBAR.bat`	El lanzador de doble clic
`E:\Euclides\PROGRESO.md`	Qué está hecho, qué falla, qué toca
`E:\Euclides\diario\`	Lo que se hizo cada día y lo que se midió